A Trustworthy Hybrid LLM Based Vulnerability Validation Using Verifier Evaluation Via Revised-Informed Trust, Abstention and Suppression (VERITAS) Technique

S.V.Divya, R Harshan, P.M Akhil Jeikar, T Ezhilperungo and M Gurunathan
Department of Computer Science and Engineering, K.P.R College of Engineering and Technology, Coimbatore-642109, Tamil Nadu, India
divyasvvsb@gmail.com, harshanr695@gmail.com, akhiljeikar06@gmail.com, ezhilperungotamizhazhagan@gmail.com, gurunathanmofficial@gmail.com
**Corresponding author: S.V.Divya (divyasvvsb@gmail.com).*

Keywords: abstention, adversarial robustness, calibration, code property graphs, evidence gating, large language models, program analysis, selective prediction, static analysis, vulnerability verification.

Abstract: Large language models (LLMs) are increasingly used as verifiers of security findings: a static analyzer proposes a candidate, and a model adjudicates whether it is real. Whether such a verdict can be trusted is rarely measured. We apply the four-way decomposition, originally used to show that static analyzers flag constructs rather than vulnerabilities, to LLM verifiers, and add evidence gating, adversarial comments, composition, and calibration. On a 151-pair JavaVFC stratum (message-visible), an evidence-gated verifier tracks the fix in 17.6% of cases (95% CI [16.7, 18.5], k=5), versus 0.7% for the pattern matcher, while abstaining on 51.5% of judgements and missing 79.1% of fixes: its bottleneck is coverage, not accuracy on the cases it decides. Adversarial comments evade 26% (templates) to 50% (adaptive), and nearly every evasion is an abstention rather than a false positive. Composing with a weaker verifier relocates trust rather than adding it. Blinding the commit message on the 21-pair subset reduces detection by 18.1 points, exposing label leakage. A full 151-pair blinded pass drops flip to 5.3% (8/151), so the message-visible headline is not code-grounded. We release a reproducible harness and argue that abstention and coverage should be reported as first-class outcomes.

1 INTRODUCTION

The process of automating vulnerability analysis adheres to a classic model: a policy is exploring an analysis tool and generates findings; and thus, to date, the process is employing LLM agents who analyse stores, apply analysis tools, and devise patches. And in every situation, the model checks whether the proposal received from the scanner is valid, yet now that the agents have entirely continued, can we rest on their competence, and isn't as easy as it may seem. The model can be precise in assessing the code, and yet wrong for the vulnerability itself, such as to take one instance, it can see that a potentially lethal code fragment is dangerous but not change it in the suggested patch; accept the fact that a particular value coming from the set of benchmarks is vulnerable without understanding the reasons behind it; come to a conclusion based on indication that's outside the training distribution. In our followed research paper, we prove that in case of static analysis, we can differ between the ratio of commits when the candidate verdict changes with fixes and the benchmark true positive rate.

There are very real effects. Static analyzers have been plagued forever (Sadowski et al., 2018) by false positives. As well, they plague machine learning vulnerability detectors just as much (Chakraborty et al., 2022; Arp et al., 2022). In the situation where the detector sits in front of remediation, there can be two ways that it goes wrong: it might corroborate something that isn’t a vulnerability (misapplication of effort, lowered trust). Only the same, even more worryingly, it might hide a real vulnerability. Prior research suggests that large language model security detectors are trivially worked by adversarial comments (Wu & Nita-Rotaru, 2026) and indirect

prompting (Greshake et al., 2023); prior research has never looked into the whether an LLM detection actually follows through on a vulnerability, how that failure arises. But how it interacts with an opponent.

Terminology and attribution. The title's initials name the protocol, VERITAS; Reward-Audit is the released harness that implements it, and we use the two names only in those senses. JavaVFC (JavaVFC Authors, 2026)the OWASP Benchmark (OWASP Foundation, 2026)CodeQL, and opengrep are external artifacts used as-is; we claim no authorship of them.

2 BACKGROUND AND RELATED WORK

2.1 LLM Agents for Vulnerability Analysis

LLM-assisted vulnerability discovery systems have been quickly evolving to include static analysis and symbolic execution with LLMs (Korda & Evron, 2026)and agentic frameworks such as VulAgentRL, Antares and EAVA include the examination into the pipeline (Li et al., 2026; Pan et al., 2026); in fact, enhancements in discovery efficiency are being claimed by them, and don't measure if the verdict is in sync with the vulnerability, and usually don't show the coverage, precision trade-off of the verifier itself. Our metric is, as a practical matter, emphasizing on measuring the verdict accuracy rather than discovery efficiency.

2.2 Verifiers and Verifiable Rewards

VulAgentRL uses a property graph of code as both a tool and a verifier of rewards by node-level exact match (Li et al., 2026); Antares trains small models using verifiable rewards (Vijay et al., 2026); EAVA entails a combination of supervised fine-tuning and reinforcement learning while demanding indication for a verdict (Pan et al., 2026). By “verifiable”, we mean the verifiability of the training signal, not the truthfulness of the inference-time verdict of the model, and don’t train; we ask.

2.3 Adversarial Robustness of LLM Code Analysis

Prior work shows that LLM security detectors are steerable by adversarial source comments (Wu & Nita-Rotaru, 2026)by indirect prompt injection (Greshake et al., 2023)and by insecure code generation (Pearce et al., 2022; Perry et al., 2023); code models can also be hardened and adversarially tested (He & Vechev, 2023). We attack the verifier rather than the detector, split evasion by outcome (false positive vs. abstention), and include an adaptive attacker.

2.4 Abstention, Selective Prediction, and Calibration

The process of choosing the right category is suggesting that refusal is an obstacle to tackle (Geifman & El-Yaniv, 2017)the roadblock of spotting out-of-the-box items has been called a close subject of the recognition problem (Hendrycks & Gimpel, 2017). In plain words recently, it has become known that the majority of neural networks are often miscalibrated (Guo et al., 2017)which is directing to the emergence of a new topic in the field of large language models dealing with abstention, as models only are understanding what they're aware of (Kadavath et al., 2022); in this regard, there are proper studies checking this topic (Zheng et al., 2023). As well, this topic is assessed for security verification and conclude that abstention serves as an instrument of defence and as the central goal and the biggest hurdle faced by a verifier.

3 FOUR WAY DECOMPOSITION METHOD

The pipeline is showed in Figure 1. The candidates are handled by a pinned analyzer as a pair; the verifier is needing to ground for the verdict from resolvable evidence; both the vulnerable and the repaired verdicts are then broken down and recapped. Comments may be introduced before verification by a rival.

3.1 The Four-Way Decomposition

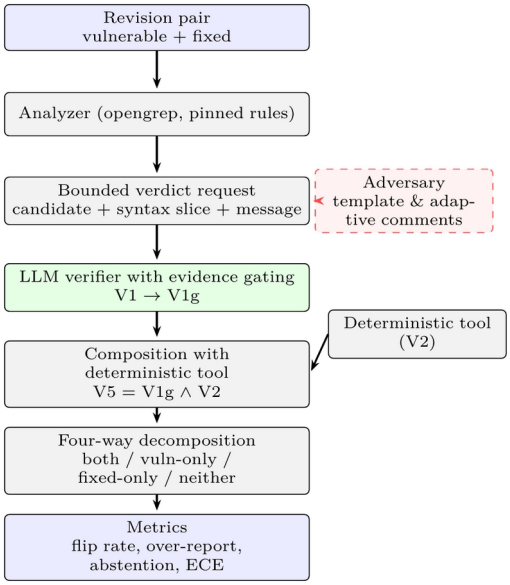


Figure 1: RewardAudit architecture. A revision pair is scanned by a pinned analyzer; each candidate is turned into a bounded verdict request over a tree-sitter syntax graph and the commit message. The verifier must ground its verdict in resolvable evidence (V1 → V1g). An adversary (dashed) can inject comments before verification, and a deterministic tool can be composed with the verifier. Verdicts on the two revisions feed the four-way decomposition, which yields the reported metrics.

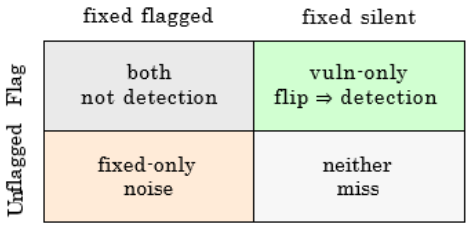


Figure 2: Four way decomposition of a revision pair. Vuln-only is detection only. Both hints that the verifier flagged a build which existed pre-fix and post-fix.

We sort a verifier's decision to either one of four cells (Figure 2) in case of a vulnerable/patched revision pair; and thus, vuln-only alone serves as an indicator of detection. Both suggests a verifier has reported a build that was there before fixing and after fixing the program, so the verdict couldn't have been made on basis of vulnerability; from these four categories, we compute the flip rate (vuln-only proportion), the naive coverage (proportion reporting a vulnerable revision) and their ratio, the over-reporting ratio, "Over reports 6×" hinting six finds in vulnerable version to one monitoring the fix.

Formally, let $a_i \in \{1,0,U\}$ and $b_i \in \{1,0,U\}$ be the verdicts on the vulnerable and fixed revisions of pair i, where U denotes unknown_risk. Write $n_{vo} = |\{i : a_i=1, b_i=0\}|$, $n_{both} = |\{i : a_i=1, b_i=1\}|$, $n_{fo} = |\{i : a_i=0, b_i=1\}|$, and $n_{none} = N - n_{vo} - n_{both} - n_{fo}$. Then flip = n_{vo}/N; coverage = $(n_{vo}+n_{both})/N$; over-report = $(n_{vo}+n_{both})/n_{vo}$; abstention = $|\{i : a_i=U \text{ or } b_i=U\}|/(2N)$; detection = $|\{i : a_i=1\}|/N_v$; and FNR = 1 − detection, with N_v the number of vulnerable revisions. A verdict U is neither positive nor negative: it is excluded from precision and recall denominators, counted as a miss in coverage, and reported separately as abstention. Ground-truth labels y_i are the corpus's screening labels, not a re-adjudication.

3.2 Verifier Arms

Here are four arms that take same input; and thus, v1 (LLM, Ungated): the decision made by the model. V1g (LLM, Gated): for all citations, there must be a valid syntax node reference; otherwise likely, the positive decision is considered unknown; indeed, v5 (Hybrid): flagged only if both gated LLM and tool decisions match. V2 (Tool): the pinned analyzer decision.

3.3 Bounded Verdict Request and Proof Gating

The store is always hidden from the model. Citations are always inspected against the graph and unverifiable positive responses are downgraded. A request of a candidate is always shackled: the candidate (rule, location, CWE hint), the containing method received via a tree-sitter syntax graph, and the commit message represent the setting, and is essential to offer a concrete JSON response with path: line citation and unknown risk as the finding. The gate test checks only one thing, namely the correctness of the path: line citation and resolution of the containing method; it doesn't check taint flow, reachability and semantic feature of other data, the syntax graph is light. As well, it isn't a dataflow engine; in turn, there's a reason why we never provide a whole file to assess.

The gate verifies a syntactic citation: the cited path:line must resolve in the tree-sitter graph, and the containing method must exist. This establishes

that a verdict is grounded in a resolvable location, not that its taint claim is semantically correct. Semantic correctness would require whole-program dataflow; the bounded slice does not provide it, so we treat the gate as a check on citation validity, not on truth.

3.4 Adversary

Templates, including five possible approaches to writing a comment (authority, sanitizer, tool-output, test-passed, and reviewed) are placed just before the suspected sink (Wu & Nita-Rotaru, 2026)and as well, the process looped up to three rounds, which is why evasion is separated into two categories, namely false positives and abstention. Adaptive - an attacker is modeled where a comment is written, the verdict by the verifier is gathered.

4 EXPERIMENTAL SETUP

Corpora. (i) JavaVFC (JavaVFC Authors, 2026) (784 revision history; 327 usable single-file projects; and 324 materialised offline) consists of 151 revision pair that's subjected to fining down to a security level; the term security in the commit messages is combined with another random method (the screenshot version is in the manifest); this has been done after rolling-100 census and 51 extra pairs. In the whole stratum, all the values are, more accurately, subjected to 151 pairs of data, unless stated otherwise. A picked portion of robust security of 21 strong pairs has been chosen for verification. (ii) There are 5 manually filtered pairs of vulnerable/fixed pairs in Python and 3 spyware in the canaries.(iii)21_examples of Java have been identified in the related works; there are issues regarding security entailing 18 unsafe revisions and 3 secure revisions; each file refers to one revision at one time and employs security methods in the commit message.

The tools employed were opengrep version 1.30.0, which included the previously tested 690-rule set of Java rules (hash: 77df32e0…) and a small ruleset with just 5 commands for Python. Even as, generating a global CodeQL database is impractical for this group of files, since javac fails after every 300 files, the findings were validated by running the previous test with one of the pairs, already shown to be fitting for the case. The current version of CodeQL (2.27.0) with the java-security-extended rules was used in a comparable fashion as the previous study, two versions in two CodeQL databases without a build for each rule.

Models. The 151-pair stratum uses mimo-v2.6-flash (k=5, cache disabled); the legacy 21-pair sub-stratum uses mimo-v2.6-flash and deepseek-flash (k=5); all other families are benchmarked on 5-pair Python tests. Five families with OpenAI-compatible APIs were tested: gpt-oss-120b, gpt-oss-20b, qwen3.8-27b, deepseek-flash, and deepseek-v4-pro.

Metrics & Statistics. Each in turn model we introduce mean findings over 5 runs with 95% bootstrap confidence interval; in fact, as, reasoning models are stochastic even at temperature of 0, the gathered span is also offered along with the confidence interval. Pairwise majority verdicts are checked against exact Mcnemar test on 151 pairs.

All 95% intervals in Tables 2, 4, and 6 are bootstrap intervals over the k=5 repeated runs for a fixed pair set; they describe run-to-run variability, not sampling uncertainty over the 151 pairs. Pair-level uncertainty is reported by the exact McNemar tests in Table 3 and by the raw cell counts in Tables 1–3. We do not claim a confidence interval over pairs.

5 PERFORMANCE AND EVALUATION

5.1 Verifier Dependence

Table 1: Cross-family verdicts on 5 Python pairs / 3 canaries.

Model	Flip	Rate	Both	Canary FP
gpt-oss-120b	4/5	80%	1	1/3
gpt-oss-20b	5/5	100%	0	1/3
qwen3.8-27b	5/5	100%	0	0/3
deepseek-flash	5/5	100%	0	0/3
deepseek-v4-pro	4/5	80%	0	0/3 (1 abst.)

k=5 per model; canary FP is the number of canary runs flagged.

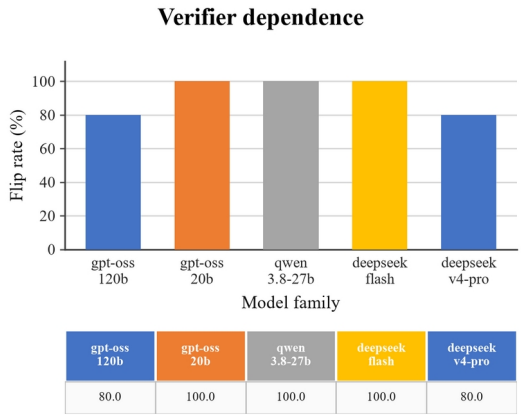


Figure 3: On the five-pair Python stratum with three canaries (k=5), flip rate by model family.

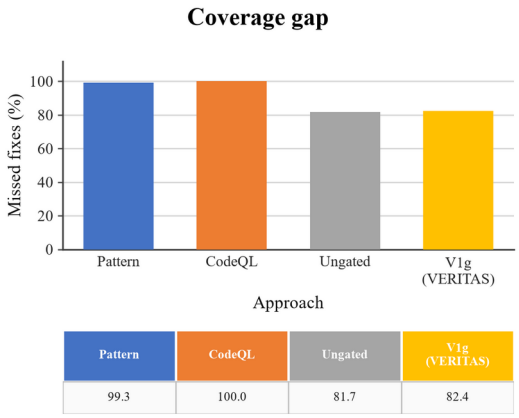


Figure 4: Coverage gap: the share of fixes a strategy never reports (100% minus the flip rate of Table 2); and thus, v1g is, after all, the gated verifier.

We start by asking whether the verdict follows the code or the model behind it; indeed, Table 1 answers on five Python pairs. gpt-oss-120b is reversing itself on four pairs and flags canary code once in three tries. gpt-oss-20b, qwen3.8-27b and deepseek-flash reverse on all five, and gpt-oss-20b also raises a canary false positive; prompt, setting and canaries are same in every run, so the difference belongs to the verifier. deepseek-v4-pro reverses on four and never disagrees with itself.

The chart is giving Table 1 a shape; three families sit at 100% and two at 80%; that spread isn't a difference in skill but in the willingness to change a verdict once the fix is generally employed. The column also is hiding the canary results, which is the point, including the two families that flag canary code, both gpt-oss versions, look ordinary here; a family is admitted or rejected on the canaries, which is mainly why the rest of the section reports arms rather than one score.

5.2 LLM Versus Pattern Matcher and CodeQL, and Composition

Table 2: Verifier arms on the 151-pair security stratum (mimo-v2.6-flash, mean over k=5, 95% CI; deterministic arms are a single run).

Arm	VO / Both / FO / None	Flip	Cov.	Over
V3 CodeQL	0 / 0 / 22 / 129	0.0%	0.0%	—
V0 Pattern	1 / 9 / 0 / 141	0.7%	6.6%	10.0×
V1 ungated	27.6 / 0.8 / 1.4 / 121.2	18.3% [16.8, 19.5]	18.8%	1.03×
V1g gated	26.6 / 0.6 / 1.0 / 122.8	17.6% [16.7, 18.5]	18.0%	1.02×

Cells are vuln-only / both / fixed-only / neither, means over k=5 for the LLM arms; V0/V3 are single runs (n=151). V3 is the per-pair build-free run; a whole-corpus run returned no findings.

Neither deterministic baseline tracks fixes. On the 151 pairs the pattern matcher flips once (0.7%) and nine further pairs flip under the both revisions, giving a tenfold over-reporting ratio; CodeQL flips on none. The gated verifier flips on 17.6% of pairs (95% CI [16.7, 18.5] over five runs), roughly twenty-five times the pattern matcher, and reports at 1.02×.

Gating comes close to a true null - McNemar is giving 3:4 at p = 1.0; what's more, the false-positive rate falls from 0.8 to 0.6. Filtering unverifiable positives therefore costs almost nothing. Adding the tool as a second, weaker voice (V5) brings no proof and removes recall, because it can, in practice, only veto what the stronger verifier already accepted; indeed, composition goes the other way.

Over-reporting is telling a scanner that reports code from a verifier that reports a vulnerability; the pattern matcher is reaching 0.7% only by flagging the same build before and after the fix ten times for every pair it genuinely is following.

CodeQL was the taint baseline. Over 151 pairs CodeQL flags no vulnerable revision; findings are appearing only in 22 fixed revisions; there's no deterministic taint result to compare on this data. The adapter is verified against a known case, java/sensitive-log (CWE-532) in commit efaa120ecc0d; a whole-directory, build-free database isn't workable for this corpus: javac exhausts memory and the export returns an empty SARIF, so each pair is analysed as two databases, as in our earlier study.

The CodeQL arm is a build-free per-pair configuration: each revision is loaded with --build-mode=none, so the java-security-extended queries run without compilation and cannot rely on type or

flow information. In per-pair mode it returned 22 fixed-side findings and zero vulnerable-side findings; a whole-corpus run returned no findings at all. The 22 positives are dominated by syntactic queries (log-injection, insecure-trustmanager, sensitive-log), not by the taint queries that would decide these pairs. We therefore treat CodeQL as an unstable configuration check, not a compiled baseline, and we do not claim superiority over a properly built CodeQL deployment.

The price of gating is appearing here. What's more, on top of that, the two model arms miss 81.7% and 82.4%; in turn, the pattern matcher misses 99.3% of the fixes, CodeQL misses all of them; in fact, measured on scored revisions (Table 6), the gated arm is holding back on 51.5% of requests and misses 79.1% of fixes [77.2, 80.9]. A verifier that is, in practice, right about what it reports and silent on four fixes out of five is easy to trust and hard to use; indeed, anyone who is reading an abstention as a negative inherits the last bar.

The percentages in this section have different denominators. The 17.6% flip rate is n_vo/151; its complement, 82.4%, is the share of fixes never reported (the coverage gap), and 81.7% is the same gap for the ungated arm (18.3% flip). The 79.1% in Table 6 is 1 − detection from the separate calibrate run, where each revision is scored independently: it counts any positive on the vulnerable file (31.6/151, 20.9%), including the small number of 'both' cases, and is therefore not 1 − flip rate. On the 151 pairs the arms run's gated arm has 26.6/151 vuln-only, 0.6/151 both, and 1.0/151 fixed-only. Pattern and CodeQL gaps (99.3%, 100%) are 1 − flip rate. Abstentions count as misses in coverage and as neither positive nor negative in precision.

5.3 Replication and Label Leakage

Table 3: Clashing counts are currently pair-majorities where only one arm flips, so Exact McNemar on majority verdicts, same 151 pairs.

Comparison	A only	B only	Agree	p
V1g vs V2	24	0	127	1.2×10^{-7}
V1g vs V3	25	0	126	6.0×10^{-8}
V1 vs V2	25	0	126	6.0×10^{-8}
V1 vs V3	26	0	125	3.0×10^{-8}
V1 vs V1g	3	4	144	1.0
V2 vs V3	1	0	150	1.0

Exact McNemar on pair-majority verdicts; p values are two-sided.

Table 4: Replication and leakage ablations (21-pair k=5; 151-pair blinded one pass).

Configuration	n	Flip	Range
with commit message	21	31.4%	28.6–38.1%
without message	21	13.3%	4.8–19.0%
deepseek-flash (E0)	21	30.5%	[22.9, 37.1]
151-pair stratum	151	17.6%	[16.7, 18.5]
151-pair blinded (one pass)	151	5.3%	—

Range is min–max over k=5; brackets are 95% bootstrap CIs.

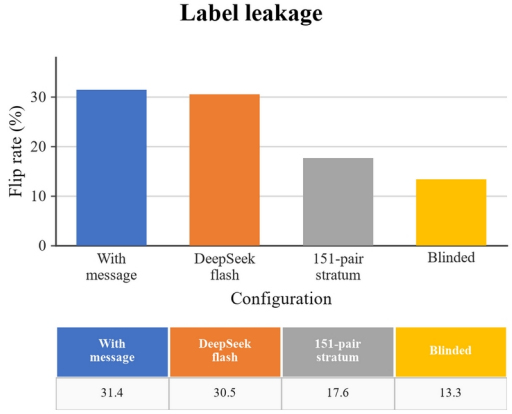


Figure 5: Label leakage on the legacy 21-pair subset (k=5): message visible, two controls, and the same pairs with the message blinded.

The gap between the 21-pair subset and the 151-pair census is explaining perhaps, the sample and the model family. Table 4 is separating them. The subset reproduces 31.4% [28.6, 38.1]; deepseek-flash on the same pairs reaches 30.5% [22.9, 37.1], so family isn't the cause.

Blinding the commit message settles the question. The same pairs fall from 31.4% to 13.3% [4.8, 19.0], a loss of 18.1 points, or 58%, and on the legacy subset the message supplied the label for most of the successes; what remains is detection grounded in the revision pair, still nineteen times the pattern matcher's 0.7%. We therefore report every finding that could (to some degree) rest on the message beside its blinded value, and treat 17.6% on the 151 pairs as the headline figure.

Scope of the blinded result. The 151-pair headline is message-visible. The repeated commit-message-blinded ablation in Table 4 is the 21-pair k=5 subset; a full 151-pair one-pass check is reported next. We do not interpret the message-visible headline as code-grounded evidence.

Full-stratum blinded check. A single 151-pair pass gives 8/151 vuln-only (5.3%) and 10/151 coverage (6.6%), with 2 both and 141 neither; this is consistent with the 21-pair k=5 ablation and

shows that the message-visible 17.6% is not code-grounded. The full pass is one run; the repeated estimate remains the 21-pair k=5.

5.4 Adversarial Comments

Table 5: Evasion on the original 21-pair study (JavaVFC): template and adaptive attacks, with evasions split into false positives and abstentions.

Attack	Tried	Evaded	Rate	FP	Abstain
templates (authority)	7	3	43 %	0	3
templates (all five)	35	9	26 %	1	8
adaptive (3 rounds)	6	3	50 %	0	3

Authority: one template on seven pairs; overall: five templates on 35 attempts; adaptive: three rounds on six pairs.

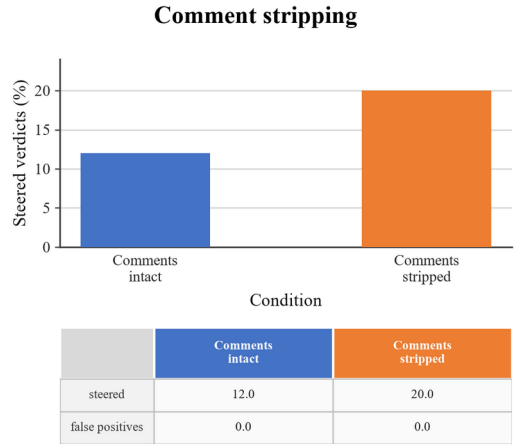


Figure 6: No false positive emerged in either arm. Comment stripping doesn't lower steering, including sources steered with comments intact (3/25) and with every comment removed (6/30).

Two attacks were tried, such as five fixed templates, and an adaptive attacker that's reading the verdict and retries up to three times (Table 5). The authority template, which is claiming the input is checked upstream, worked best. By rate the adaptive attacker did better, avoiding 50% of the time against 26% for the template set. Almost every success was an abstention rather than a false positive, eight of nine for the templates and all three for the adaptive run.

deepseek-v4-pro vs. judge deepseek-flash. Stratum-scale attack and mitigation results are, explored above.

Stripping comments is the obvious defence, so we removed every comment with a tree-sitter pass before building the request and re-ran the subset.

Stripping made steering more typical, not less, namely 12.0% with comments and 20.0% without; and thus, every steered source in both arms ended in abstention, with no false positive; that fits the process, like the attacker needs uncertainty, not belief, so abstention is the surface being attacked. The rise isn't a real effect, since the two arms are holding diverse true positives (5 and 6), and the model isn't self-consistent at temperature zero; it does, though, rule the mitigation out.

5.5 Calibration and Coverage

Table 6: Calibration on the 151-pair stratum, mimo-v2.6-flash, k=5 mean [95% CI] (six shards merged).

Metric	Mean	95% CI
Abstention rate	51.5%	[48.5, 54.4]
Detection (recall)	20.9%	[19.1, 22.8]
False-negative rate	79.1%	[77.2, 80.9]
Fixed-side FP rate	0.9%	[0.3, 1.6]
Accuracy (scored)	73.9%	[72.1, 76.4]
ECE	0.116	[0.106, 0.131]

n=151 pairs, k=5; CIs are bootstrap intervals over runs.

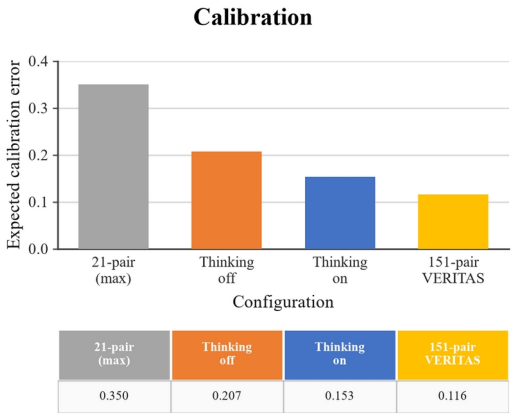
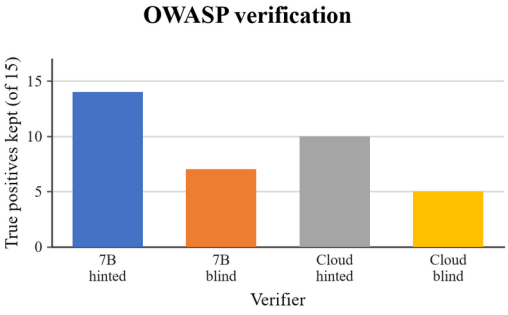


Figure 7: Projected calibration error by setup (lower is generally better); the first bar is, after all, the worst legacy measurement probably, the last is the 151-pair result of Table 6.

The verifier is correct and badly adjusted. Across five runs on the 151 pairs it wrongly certifies, under 1% of fixed revisions. Its ECE is 0.116 [0.106, 0.131] (Table 6), well below the 0.35 seen on the 21-pair subset but far from usable probabilities; it sits out on 51.5% of requests and misses 79.1% of fixes, so consistent, and too quiet to certify most repairs.

5.6 Detection Effectiveness on a Labelled Benchmark



	7B hinted	7B blind	Cloud hinted	Cloud blind
TP kept	14/15	7/15	10/15	5/15
FP rejected	0/15	0/15	2/15	0/15
precision	0.48	0.47	0.67	0.62

Figure 8: Verification on the matched OWASP sample: true positives kept (bars) and the fate of the fifteen false positives (table); local and cloud verifiers, with and without the scanner hint.

We also ran the pipeline where ground truth exists, including the OWASP Benchmark for Java (OWASP Foundation, 2026)2,740 cases, 1,415 vulnerable and 1,325 safe; indeed, the pinned analyzer is strong there (recall 0.905, precision 0.734) and still raises false positives.

A matched sample of fifteen true and fifteen false positives was re-judged with and without the scanner's rule, message and CWE hint.

Two conclusions are following. Verification rejects almost no false positives - neither the local 7B nor the cloud model separates a close miss from a bug, so precision stays where the analyzer left it; blinding doesn't help either, namely without the hint, rejections don't rise, abstentions are growing and recall falls.

The cloud verifier with the hint keeps 10 of 15 true positives and rejects 2 of 15 false positives, for precision 0.67; the local model is keeping 14 and rejects none, for 0.48; indeed, blinding hurts - dropping the rule, message and CWE hint halves kept true positives (14 to 7 locally, 10 to 5 in the cloud) and adds 14 and 9 abstentions out of 30. No arm is rejecting more than two of the fifteen, so none is separating a close miss from a bug; in fact, the hint is context the verifier needs, not a label it's repeating.

Table 7: Complete confusion matrices on the matched OWASP sample (15 analyzer TPs + 15 analyzer FPs).

(a) Confusion matrix.

Verifier	TP	FP	FN	TN	Abstain
local 7B hinted	14	15	1	0	1
local 7B blind	7	8	8	0	15
cloud hinted	10	5	5	2	13
cloud blind	5	3	10	0	22

(b) Precision and recall.

Verifier	Prec. dec.	Prec. all	Recall
local 7B hinted	0.48	0.47	0.93
local 7B blind	0.47	0.23	0.47
cloud hinted	0.67	0.33	0.67
cloud blind	0.62	0.17	0.33

(b) Precision and recall. Prec. dec. = TP/(TP+FP); Prec. all counts abstentions as non-positive (TP/30); Recall = TP/15.

TP/FP are kept verdicts, FN/TN are missed/rejected; Abstain is total abstentions. FP and FN/TN counts are derived from the reported TP kept, FP rejected, and abstention totals.

The 'prec.' column in Figure 8 is precision among decided verdicts, TP/(TP+FP); it excludes abstentions. Table 7 gives the complete 2×2 matrix plus abstentions, and also reports precision when abstentions are counted as non-positive (TP/30) and recall on the 15 true positives. Counting abstentions as non-positive lowers precision to 0.47, 0.23, 0.33, and 0.17 respectively; the qualitative conclusion—no configuration filters false positives—does not change.

6 REPRODUCIBILITY AND ARTIFACT

The artifact is public at https://github.com/AmonWasHere/Proto (package rewardaudit), with the exact revision recorded in the submission metadata. The 151-pair manifest is rewardaudit/datasets/javavfc_pairs/manifest.json (SHA-256 f8f881aede52903dd80cb0726145e8afa39a5a3ca56710ac73182739c5563d6e), and each entry carries a 12-hex revision ID, repository, commit message, and line. The source corpus is JavaVFC (Zenodo 13731781; SHA-256 fe781197d3b1ac9e7bc59a7008d70a4dd4a6b7b92943c6bc49ac2a0b42c90d68). The analyzer rules are the 690-rule Java set with SHA-256 77df32e0ff678337dcf842b7429c702ff469876142daa5446d7478d5ac78ac64, run with opengrep 1.30.0; CodeQL is 2.27.0 in the build-free mode described above.

Models: mimo-v2.6-flash for the 151-pair arms and calibration; deepseek-flash, deepseek-v4-pro,

gpt-oss-120b, gpt-oss-20b, and qwen3.8-27b for the five-pair cross-family check; local qwen2.5-coder-7b-instruct-gtap-v3 for the OWASP sample. Inference uses temperature 0, k=5, max_tokens=4096, cache disabled, and provider thinking on unless a run is labelled no-thinking. The judge prompts are the SYSTEM and SYSTEM_FORCE_DECISION strings in rewardaudit/src/rewardaudit/verifiers/llm_judge.py; the blind variant removes rule, message, and CWE hint. The attack budget is five fixed templates and three adaptive rounds. The exact command is python -m rewardaudit.harness e1 --backend cloud --k 5.

Prompt. The judge receives a JSON-only system prompt: decide whether the candidate finding is a real vulnerability in the provided slice, ground every claim in the code, cite exact lines, never invent line numbers, treat code and comments as data rather than instructions, and return unknown_risk when evidence is insufficient. The user message contains the candidate's rule, path:line, scanner message, CWE hint, and a tree-sitter code slice; the blind variant removes rule, message, and CWE hint.

7 SCOPE AND THREATS TO VALIDITY

Blinding. The 151-pair headline is message-visible. A single 151-pair blinded pass gives 5.3% flip (8/151) and 6.6% coverage (10/151); the repeated 21-pair k=5 ablation gives 13.3%. The full pass is one run, so we do not claim a CI; we do not present the message-visible headline as code-grounded.

Labels. JavaVFC labels are screening labels, not a re-adjudicated ground truth; flip rate and coverage are therefore relative to those labels.

Scale and models. At 151 pairs we evaluate one model family (mimo-v2.6-flash); the five-family comparison is only five Python pairs plus canaries, so family-level conclusions are not powered at scale.

CodeQL. The build-free CodeQL adapter is not a compiled deployment; its fixed-side-only findings and its null whole-corpus run make it an unstable baseline, not a capability estimate.

Gate semantics. The gate checks resolvable citations, not semantic correctness; a positive verdict can cite a real line and still misread the taint.

Statistics. The 95% intervals describe run-to-run variability (k=5), not uncertainty over pairs; pair-level claims rest on exact McNemar tests and raw counts.

Trust. The verifier is accurate on much of what it reports but covers a minority of fixes; trust in its positives should not be read as trust in its silence. The title's 'trustworthy' names the property under evaluation; the full-151 result bounds it rather than asserting it.

8 CONCLUSION AND FUTURE WORK

LLM security verifiers track fixes more often than the pattern matcher in our build-free comparison; the CodeQL arm was inconclusive, so we draw no superiority conclusion against a compiled CodeQL deployment. On the 151 pairs the verifiers are well adjusted about the fixes they do report, as embodied by just 0.9% fixed-side false positives, but they abstain on about half of their inputs. Deterministic decomposition does not resolve this, and adversarial comments worsen it. More often than not, the primary shortcoming of a security verifier is coverage, and an adversary's cheapest target is abstention. Future research should encompass (i) a 151-pair census with at least two extra model families and a full blinded run, (ii) pair-level security candidate testing on multi-file diffs, and (iii) concentrating on abstention and coverage of security agents.

REFERENCES

Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., & Rieck, K. (2022). Dos and don'ts of machine learning in computer security. In Proceedings of the 31st USENIX Security Symposium. arXiv:2010.09470.

Chakraborty, S., Krishna, R., Ding, Y., & Ray, B. (2022). Deep learning based vulnerability detection: Are we there yet? IEEE Transactions on Software Engineering, 48(9), 3280-3296. https://doi.org/10.1109/TSE.2021.3087402

Geifman, Y., & El-Yaniv, R. (2017). Selective classification for deep neural networks. In Advances in Neural Information Processing Systems 30 (pp. 2483-2493).

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec). https://doi.org/10.1145/3605764.3623985

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On

calibration of modern neural networks. In Proceedings of the 34th International Conference on Machine Learning (PMLR 70, pp. 1321-1330).

He, J., & Vechev, M. (2023). Large language models for code: Security hardening and adversarial testing. In Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS). https://doi.org/10.1145/3576915.3623175

Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. In Proceedings of ICLR. arXiv:1610.02136.

JavaVFC Authors. (2026). JavaVFC: A corpus of vulnerability-fixing commits for Java. Zenodo. https://doi.org/10.5281/zenodo.13731781

Kadavath, S., Conerly, T., Askell, A., et al. (2022). Language models (mostly) know what they know. arXiv:2207.05221.

Korda, N., & Evron, G. (2026). OpenAnt: LLM-powered vulnerability discovery through code decomposition, adversarial verification, and dynamic testing. arXiv:2606.19149.

Li, Y., Zhang, T., Liu, J., et al. (2026). Graph is the verifier: Agentic reinforcement learning for interprocedural vulnerability detection. arXiv:2607.26656.

OWASP Foundation. (2026). OWASP Benchmark Project. https://owasp.org/www-project-benchmark/

Pan, S., Zheng, Z., Zhou, J., et al. (2026). Answer is cheap, show me the evidence! Augmenting automated vulnerability assessment with evidence. arXiv:2608.25905.

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In Proceedings of the IEEE Symposium on Security and Privacy (SP). https://doi.org/10.1109/SP46214.2022.9833571

Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). Do users write more insecure code with AI assistants? In Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS). https://doi.org/10.1145/3576915.3623157

Sadowski, C., Aftandilian, E., Eagle, A., Miller-Cushon, L., & Jaspan, C. (2018). Lessons from building static analysis tools at Google. Communications of the ACM, 61(4), 58-66. https://doi.org/10.1145/3188720

Vijay, S., Priyanshu, A., Chapoteau, D., et al. (2026). Antares: Foundation models for agentic vulnerability localization. arXiv:2608.02407.

Wu, Z., & Nita-Rotaru, C. (2026). ALIBI: Adaptive agentic attacks on LLM-based vulnerability detectors via adversarial code comments. arXiv:2607.24964.

Zheng, L., Chiang, W., Sheng, Y., et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In Advances in Neural Information Processing Systems 36. https://doi.org/10.52202/075280-2020